

---

# Learned Relay Representations for Forward-Thinking Discrete Diffusion Models

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

1 When Masked Diffusion Models (MDMs) generate sequences through iterative  
2 refinement, the rich internal computation over masked positions is discarded—  
3 forcing every subsequent refinement step to recompute the valuable internal  
4 information stored as model representations. To avoid a hard reset between denois-  
5 ing rounds, we propose Learned Relay Representations (RELAY), a method that  
6 allows MDMs to be “forward-thinking” when denoising—*explicitly learning how*  
7 *to propagate latent information for the benefit of future denoising steps*. RELAY  
8 introduces a differentiable per-token channel that passes information between  
9 forward passes and is trained via truncated backpropagation through time (BPTT).  
10 We show that this framework can be scaled to state-of-the-art Diffusion Language  
11 Models (DLMs), and is seamlessly compatible with techniques like block diffusion  
12 and KV caching. We first provide a thorough justification of the design choices  
13 in RELAY on a challenging Sudoku-based planning task. We then scale RELAY to  
14 Fast-dLLM v2, a state-of-the-art DLM, outperforming standard supervised  
15 finetuning on coding tasks by up to 3.7% in accuracy and 32% in inference latency.  
16 Our empirical results demonstrate that state-of-the-art DLMs can be explicitly  
17 trained to *relay* latent information forward across decoding steps, advancing the  
18 performance-latency Pareto frontier. We provide code for all our experiments.<sup>1</sup>

## 19 1 Introduction

20 Masked Diffusion Models (MDMs) generate discrete sequences via iterative denoising [Austin et al.,  
21 2021, Campbell et al., 2022, Sahoo et al., 2024, Shi et al., 2024]: starting from a fully masked canvas,  
22 each forward pass unmask a fraction of the remaining positions. The Transformer computes hidden  
23 states at every position—including those still masked—but discards them at the end of each step,  
24 beginning the next pass from the partially unmasked sequence alone. We call this the *hard reset*  
25 problem: the only information that persists across steps is the discrete tokens just committed, leaving  
26 MDMs with no way to accumulate intermediate continuous computation.

27 This matters because recurrent computation—unrolling a fixed-parameter model across many  
28 steps—is exactly the structural property that recent work has tied to improved performance on difficult  
29 reasoning tasks, as it effectively expands the function class the model can approximate [Gatmiry  
30 et al., 2024, Saunshi et al., 2025, Li et al., 2024]. MDMs already perform many forward passes per  
31 generation; the hard reset is what prevents any of that compute from being reused.

**This raises a natural question:** *How can the sequential unmasking structure of MDMs support recurrent computation that carries richer information across steps?*

---

<sup>1</sup>Anonymized code is available at: <https://anonymous.4open.science/r/relay-1D24>

Our answer is Learned Relay Representations (RELAY), a method that makes discrete diffusion models *forward-thinking*: at each denoising step, alongside any newly unmasked tokens, the model carries its last-layer hidden states forward as a learned *relay*, giving the next forward pass direct access to the prior step’s continuous computation. Simply piping these states forward, however, does not by itself ensure they encode anything useful for what follows. RELAY therefore trains the relay end-to-end with truncated backpropagation through time (BPTT), shaping it to be maximally informative for the next several denoising steps and enabling a form of latent chain-of-thought across the unmasking trajectory.

**Contributions.** We introduce RELAY, which equips MDMs with learned relay representations—continuous latent states passed forward across decoding steps and trained end-to-end via truncated BPTT. RELAY is architecture-agnostic and leaves the inference-time decoding procedure of MDMs (unmasking schedule, sampling) unchanged; the only addition at inference is forwarding the relay alongside the committed tokens. It is also compatible with prevalent DLM acceleration techniques, including block diffusion [Arriola et al., 2024] and KV caching [Wu et al., 2025a,b].

To summarize, our key contributions are as follows:

1. We propose RELAY, a general method for incorporating recurrent computation in MDMs by training the model—via truncated BPTT—to pass a learned latent relay forward across decoding steps. RELAY can train an MDM from scratch or adapt a pre-trained MDM through lightweight adaptation.
2. We validate RELAY at LLM scale through full-parameter adaptation of Fast-dLLM v2 1.5B [Wu et al., 2025b] on coding tasks, achieving improvements of up to 3.7% in accuracy and 50% reduction in latency.
3. We perform extensive ablations that map out the design space of RELAY and validate our choices.

## 2 Background: Masked Diffusion Models

We tackle the hard reset problem in masked diffusion models by training them to pass along a learned relay state. Before presenting our approach, RELAY, we review the training and inference procedure for Masked Diffusion Models (MDMs) [Shi et al., 2024, Sahoo et al., 2024] that RELAY builds upon.

**Notation.** We denote the vocabulary as  $\mathcal{V}$ , including the  $[M]$  token. The space of sequences of length  $L$  over the vocabulary is  $\mathcal{V}^L$ . Superscripts denote the position in the sequence, e.g.,  $x^i$  is the  $i$ -th token in the sequence  $\mathbf{x} \in \mathcal{V}^L$ .  $\mathcal{M}(\mathbf{x}) \subseteq [L]$  denotes the set of masked positions in the sequence  $\mathbf{x}$ .

**Training.** The noising process proceeds by sampling a time  $t \in [0, 1]$  and masking each position in a clean sequence  $\mathbf{x}_0 \in (\mathcal{V} \setminus \{[M]\})^L$  independently with probability  $\alpha_t$ , to obtain the noised (partially masked) sequence  $\mathbf{x}_t$ . The coordinate-wise posterior distribution  $\mathbb{P}(X_0^i = x_0^i \mid \mathbf{X}_t = \mathbf{x}_t)$  is denoted as  $p(x_0^i \mid \mathbf{x}_t)$ . As noted in Zheng et al. [2024], this posterior depends on  $\mathbf{x}_t$  only through its masked pattern and revealed tokens, not on the time  $t$  itself. The coordinate-wise posterior is parameterized by a neural network denoted as  $p_\theta^i(\cdot \mid \mathbf{x}_t) \in \Delta$  for  $i \in \mathcal{M}(\mathbf{x}_t)$  and is trained by minimizing the weighted sum of cross-entropy losses for each masked position<sup>2</sup>

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_0, t, \mathbf{x}_t} \left[ \frac{1}{t} \sum_{i: x_t^i = [M]} -\log p_\theta^i(x_0^i \mid \mathbf{x}_t) \right]. \quad (1)$$

The coordinate-wise parametric posterior is implemented using embedding  $\text{EMB}_\theta : \mathcal{V} \rightarrow \mathbb{R}^d$ , unembedding  $\text{UNEMB}_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^{|\mathcal{V}|}$ , and a transformer backbone  $f_\theta : \mathcal{V}^L \rightarrow \mathbb{R}^{L \times d}$  that produce the posterior distribution:

$$p_\theta^i(w \mid \mathbf{x}_t) = \frac{e^{\ell^i(w)}}{\sum_{w' \in \mathcal{V}} e^{\ell^i(w')}}, \text{ where } \ell^i(w) = \text{UNEMB}_\theta(f_\theta(\text{EMB}_\theta(\mathbf{x}_t)))_w^i.$$

**Inference.** Generation proceeds along a decreasing time grid  $1 = t_0 > t_1 > \dots > t_K = 0$ , iteratively unmasking positions from the all-masked sequence  $\mathbf{x}_{t_0} = ([M], \dots, [M])$  to a fully unmasked sequence  $\mathbf{x}_{t_K} \in (\mathcal{V} \setminus \{[M]\})^L$ . At each step  $k$ , given the current partially masked sequence  $\mathbf{x}_{t_k}$ , the model computes logits  $\ell_k$  for the per-position posterior distribution for each masked position  $i \in \mathcal{M}(\mathbf{x}_{t_k})$  and token  $w \in \mathcal{V}$ . An unmasking policy  $u(\cdot \mid \ell_k, \mathbf{x}_{t_k})$ , which may

<sup>2</sup>We have assumed a linear noise schedule.

74 be stochastic, then selects a set of positions  $\mathcal{U}_k \subseteq \mathcal{M}(\mathbf{x}_{t_k})$  to reveal, producing the next partially  
 75 masked sequence  $\mathbf{x}_{t_{k+1}}$ . Common choices for  $u(\cdot | \ell_k, \mathbf{x}_{t_k})$  include unmasking a fixed fraction of  
 76 the remaining masks at each step [Nie et al., 2025] and confidence-based rules [Ben-Hamu et al.,  
 77 2025, Kim et al., 2025, Patel et al., 2025].

78 **The Hard Reset Problem.** After each inference step, MDMs discard the entire computational  
 79 state used to choose the newly revealed tokens. The next step starts again from  $\mathbf{x}_{t_{k+1}}$  alone. Thus,  
 80 standard MDM inference treats each partially masked sequence as a fresh prediction problem—a *hard*  
 81 *reset*—rather than as a continuation of an ongoing computation. Because models can only perform  
 82 a constant number of FLOPs in each forward pass, hard reset prevents the model from amortizing  
 83 reasoning across steps effectively. In the next section, we propose our solution to this problem: we  
 84 learn a continuous latent state that is passed across the steps of MDM inference.

### 85 3 Learned Relay Representations

86 To address the hard reset problem, we introduce a continuous differentiable state that is carried across  
 87 MDM inference steps to circumvent the *hard reset* problem.

#### 88 3.1 Augmented State Trajectories

89 The training of MDMs proceeds by sampling a data point  $\mathbf{x}_0 \sim p_{\text{data}}$ , a time  $t \sim \mathcal{U}(0, 1)$ , and a  
 90 partially masked sequence  $\mathbf{x}_t$  under the noise schedule given the time  $t$  and the data point  $\mathbf{x}_0$ . During  
 91 inference, we have a discretized time grid  $1 = t_0 > \dots > t_n = 0$ , and the corresponding inference  
 92 trajectory  $\mathbf{x}_{t_0}, \dots, \mathbf{x}_{t_n}$  obtained by using some unmasking policy  $u$ , where  $\mathbf{x}_{t_0} = \{[\text{M}]\}^L$ . We wish  
 93 to pass a continuous state forward across decoding steps, that can carry intermediate computations  
 94 from the previous step which have not yet been realized as a decoded token. We can break down this  
 95 behavior into two primitives: a model must produce a relay state  $\mathbf{h}_{t_k}$  at inference step  $t_k$ , and learn to  
 96 consume that relay state in inference step  $t_{k+1}$ . Figure 1 shows a schematic of the augmented state  
 97 trajectory produced by the model, where  $\mathbf{s}_{t_k} = (\mathbf{x}_{t_k}, \mathbf{h}_{t_k})$  is the augmented state at time  $t_k$ .

#### 98 3.2 Training

99 Since we only care about the terminal state  $\mathbf{x}_{t_n}$ , we continue to provide supervision using the  
 100 same cross-entropy loss as in standard MDMs, and train the model to produce useful  $\mathbf{h}_t$  that help  
 101 improve predictions  $K$  steps ahead using truncated BPTT. Specifically, instead of sampling  $\mathbf{x}_t$  as  
 102 in standard MDMs, we start from an all masked sequence  $\mathbf{x}_{t_0} = \{[\text{M}]\}^L$ , and perform a rollout to  
 103 obtain the augmented state trajectory  $(\mathbf{x}_{t_0}, \mathbf{h}_{t_0}), \dots, (\mathbf{x}_{t_n}, \mathbf{h}_{t_n})$ , where the total loss is the sum of  
 104 the cross-entropy losses over the entire trajectory.

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_0} \sum_{k=0}^n \log p_{\theta}^i(x_0^i | \mathbf{x}_{t_k}, \mathbf{h}_{t_k}) \quad (2)$$

105 Here  $p_{\theta}^i(x_0^i | \mathbf{x}_{t_k}, \mathbf{h}_{t_k})$  is the predicted posterior probability for the  $i$ -th position at time  $t_k$ , computed  
 106 from the augmented state  $(\mathbf{x}_{t_k}, \mathbf{h}_{t_k})$ . The relay state is an internal model state produced by the  
 107 rollout, rather than an externally observed conditioning variable. The relay state is part of the  
 108 computational trajectory, not part of the generated object. At inference time, each step carries forward  
 109 the realized pair  $(\mathbf{x}_{t_k}, \mathbf{h}_{t_k})$ , but only  $\mathbf{x}_{t_k}$  is eventually decoded into text. Thus  $\mathbf{h}_{t_k}$  serves as a  
 110 differentiable memory channel for future predictions, while the training target remains the same  
 111 token-level posterior used by standard MDMs.

112 **Constructing rollouts.** In order to perform truncated BPTT, we need to construct rollouts of  
 113 the augmented state trajectory under an unmasking policy  $u$ . Given the current augmented state  
 114  $(\mathbf{x}_{t_k}, \mathbf{h}_{t_k})$ , one step of rollout proceeds as follows:

- 115 • **Unmasking:** Sample unmasking positions  $\mathcal{U} \sim u(\cdot | \ell_k, \mathbf{x}_{t_k})$ .
- 116 • **Token Selection:** Teacher force  $\mathcal{U}$  to the values in  $\mathbf{x}_0$ , i.e.,  $x_{t_{k+1}}^i = x_0^i$  for  $i \in \mathcal{U}$ .

117 For token selection, we need to use teacher forcing because using the model’s own posterior  
 118 distribution for token selection would lead to mistakes which the model has no mechanism to correct.  
 119 Interestingly, this scheme leaves the ideal minimizer unchanged from the standard MDM training



134 For a two-step truncation beginning at step  $k$ , we have  $\lambda_{k+2} = 0$ , so the only downstream adjoint is

$$\lambda_{k+1} = (\partial_{\mathbf{h}_{k+1}} \ell_{k+1})^\top \nabla_{\ell_{k+1}} L_{k+1}(\ell_{k+1}, \mathbf{x}_0). \quad (7)$$

135 The two-step gradient is therefore

$$\nabla_\theta (L_k + L_{k+1}) = \underbrace{\sum_{j=k}^{k+1} (\partial_{\ell_j} \ell_j)^\top \nabla_{\ell_j} L_j(\ell_j, \mathbf{x}_0)}_{\text{direct gradient from immediate cross-entropy}} + \underbrace{(\partial_{\mathbf{h}_{k+1}} \ell_{k+1})^\top \lambda_{k+1}}_{\text{BPTT through relay state}}. \quad (8)$$

136 Thus, each step receives the local cross-entropy gradient through its logits  $\ell_k$ , while downstream  
137 gradients are back-propagated only through the differentiable relay path  $\mathbf{h}_k \rightarrow \mathbf{h}_{k+1}$ .

138 The final training algorithm is presented in Algorithm 1.

## 139 4 Experiments

140 Through our experiments we seek to address the following research questions:

- 141 **RQ1** Does training to be forward-thinking with BPTT improve performance and latency?
- 142 **RQ2** Does weight-tying  $\text{EMB}_\theta$  and  $\text{UNEMB}_\theta$  have an impact on RELAY, since  $f_\theta$  at the first  
143 layer must learn to consume the  $\text{UNEMB}_\theta$ -aligned relay  $\mathbf{h}$  from the last layer?
- 144 **RQ3** Can we efficiently adapt state-of-the-art DLMs to use relay representations and improve  
145 their performance-latency frontiers with negligible additional training FLOPs?

146 We first motivate the design choices for RELAY with a thorough ablation on Sudoku. Subsequently, we  
147 post-train Fast-dLLM v2 [Wu et al., 2025b], a state-of-the-art DLM, demonstrating the effectiveness  
148 of RELAY on model adaptation for DLMs.

### 149 4.1 Sudoku

150 **Dataset.** The objective of a Sudoku puzzle is to fill in a 9x9 board (of nine 3x3 sub-squares) with  
151 digits 1-9 such that each row, column, and 3x3 square contains all the nine unique digits. A puzzle  
152 has a minimum of 17 clues, which is a mathematical prerequisite for it to have a unique solution  
153 [McGuire et al., 2014].

154 **Setup.** We choose the Sudoku-Extreme<sup>3</sup> dataset [Wang et al., 2025a] as a challenging benchmark  
155 that allows us to focus on modeling choices without the risk of overfitting. For the experiments in  
156 Figure 2 we evaluate on the first 50k examples of the test split, which are representative in difficulty  
157 (Appendix C.3). All methods use the same small Transformer backbone ( $\sim 7\text{M}$  parameters; full  
158 architecture in Appendix C) with rotary position embeddings and are trained to convergence, in  
159 line with our experimental protocol of comparing methods by their test-time performance versus  
160 latency frontiers. Our predictor uses top-probabilities as confidence values  $c_i$ , sorts by increasing  
161  $1 - c_i$ , and unmask all positions whose cumulative confidence falls below a threshold  $\tau$ , falling  
162 back on the argmax if no such position exists. For RELAY’s on-policy training rollout we use a  
163 stochastic threshold  $\tau \sim \mathcal{N}(\mu = 0.15, \sigma = 0.1)$  for robustness (the threshold is a hyperparameter  
164 of the sampling decision  $\mathcal{U} \sim u(\cdot \mid \ell, \mathbf{z})$  in Algorithm 1 line 10).

165 **Baselines and ablations.** We compare four training objectives that progressively turn on the  
166 components of Algorithm 1, each instantiated with both *tied* and *untied* embeddings (whether  $\text{EMB}_\theta$   
167 and  $\text{UNEMB}_\theta$  share weights). **MLM** [Sahoo et al., 2024, Shi et al., 2024] is standard uniform masked  
168 diffusion: a single forward pass per training step ( $K=1$ ), no relay ( $R_\theta \equiv 0$ , so  $\mathbf{h} \leftarrow f_\theta(\text{EMB}_\theta(\mathbf{z}))$ ),  
169 and the on-policy sampler  $u(\cdot \mid \ell, \mathbf{z})$  replaced by a uniform random one. **Rollout** unrolls  $K=2$   
170 inner steps with the same on-policy sampler  $u(\cdot \mid \ell, \mathbf{z})$  used by RELAY, but still keeps  $R_\theta \equiv 0$  so  
171 each step recomputes  $\mathbf{h}$  from  $\text{EMB}_\theta(\mathbf{z})$  alone; this isolates the contribution of *which* positions get  
172 committed between forward passes (a related rollout training procedure is studied by Kim et al.,  
173 2026). **RELAY (sg)** additionally enables the relay path  $R_\theta(\mathbf{h})$  inside the inner loop but stop-gradients

<sup>3</sup><https://huggingface.co/datasets/sapiointinc/sudoku-extreme>

174  $h$  before feeding it back, so the backbone receives no temporal credit across the  $K$  steps. **RELAY** is  
 175 the full method:  $K=2$  BPTT through the relay ([Algorithm 1](#)). At inference we sweep deterministic  
 176 thresholds  $\tau \in \{0.05, 0.10, 0.15, 0.20, 0.25\}$  and trace each method’s accuracy-NFE frontier; lower  
 177  $\tau$  commits fewer cells per forward pass and so spends more NFEs.

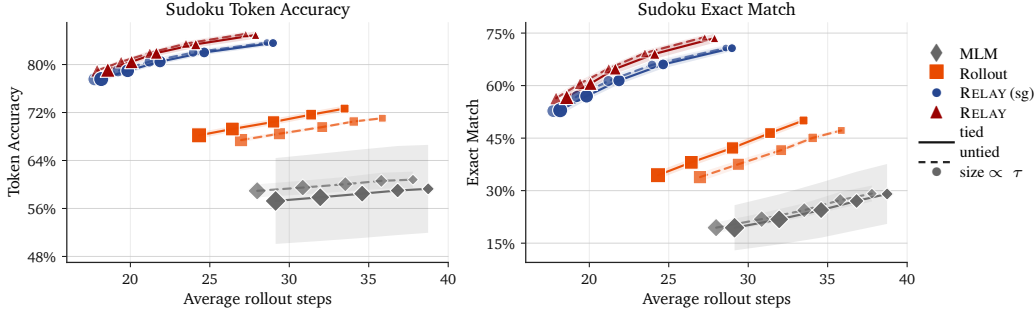


Figure 2: Accuracy-NFE frontier on Sudoku-Extreme validation. Each curve traces a single training method as we sweep the inference confidence threshold  $\tau \in \{0.05, 0.10, 0.15, 0.20, 0.25\}$ . A lower  $\tau$  commits fewer cells per forward pass and so spends more NFEs (rightward), and vice-versa. Shaded ribbons denote  $\pm 1$  sample standard deviation across three training seeds.

178 **Results and analysis.** Figure 2 plots validation metrics at the latest checkpoint for each seed.  
 179 Replacing uniform masking (MLM) with the on-policy confidence-thresholded sampler under teacher  
 180 forcing of the unmasked values (**Rollout**) yields the first improvement<sup>4</sup>. Turning the relay channel on  
 181 (**RELAY (sg)**) contributes the next big jump, highlighting the importance of a soft state carried between  
 182 forward passes. Finally, replacing the stop-gradient with  $K=2$  BPTT through the relay (**RELAY**,  
 183 [Algorithm 1](#)) yields a further separation and the best accuracy-NFE frontier across thresholds.

184 We are able to trace this last separation of RELAY over RELAY (sg) to the fact that, at the same  
 185 threshold  $\tau$ , RELAY **commits more cells per forward pass while keeping the partial board legal**  
 186 — where a board is *legal* when no row, column, or  $3 \times 3$  box yet contains a repeated digit. Legality  
 187 is a necessary (but not sufficient) condition for correctness, and is well-defined at every intermediate  
 188 denoising step, not only at the end. Since the studied architectures cannot perform recursive search,  
 189 we restrict this qualitative analysis to a *deduction-only* cohort of 2,000 test puzzles whose solutions  
 190 require only human-like deduction strategies (Advanced or Master heuristics; cohort construction  
 191 detailed in Appendix C.3).

192 At the matched threshold  $\tau = 0.15$ , RELAY produces a fully legal final board 74.8% of the time  
 193 versus 70.7% for RELAY (sg) (+4.1 pp), and incurs  $-15\%$  as many row/column/box violations across  
 194 the rollout (0.90 vs. 1.06 per puzzle); these legality gains are uniform across the Advanced (+4.0 pp)  
 195 and Master (+4.1 pp) strata. In other words, **BPTT teaches the relay to keep the partial board self-**  
 196 **consistent under more aggressive unmasking**: at the same confidence threshold  $\tau$ , RELAY commits  
 197 more cells per forward pass while still honoring the row/column/box constraints, so the rollout reaches  
 198 the same accuracy in fewer total forward passes — producing the strict outward shift of the ( $\tau \rightarrow$   
 199 NFE, accuracy) frontier in Figure 2. Table 1 reports the corresponding exact match and mean NFE  
 200 at  $\tau = 0.15$  on both the unfiltered test split and the deduction-only cohort: RELAY attains the highest  
 201 exact match and the lowest mean NFE in every (slice, tying) cell, with a +4 to +6 pp gain over RELAY  
 202 (sg) at uniformly lower NFE. Tying versus unttying  $\text{EMB}_\theta$  and  $\text{UNEMB}_\theta$  has only a marginal effect  
 203 on any objective ( $\leq 3$  pp exact match across all rows), consistent with the small Sudoku vocabulary  
 204 leaving the residual stream ample capacity to carry both predictive and relay-bearing information.

## 205 4.2 Pretrained Model Adaptation: Fast-dLLM v2

206 Next, we investigate whether state-of-the-art DLMs can be efficiently adapted into RELAY diffusion  
 207 models with a limited amount of finetuning, and whether this adaptation can improve their accuracy-  
 208 latency frontiers.

<sup>4</sup>Preliminary experiments without teacher-forcing introduced exposure bias and degraded performance significantly, consistently with the observations of Kim et al. [2026].



| Objective  | Tying | Acc (%)               |                    | Mean NFE              |                    |
|------------|-------|-----------------------|--------------------|-----------------------|--------------------|
|            |       | unfiltered test       | unfiltered test    | deduction-only        | deduction-only     |
| MLM        | ✓     | 20.27% ± 0.25%        | 13.76 ± 0.56       | 32.23% ± 1.87%        | 9.27 ± 0.17        |
| MLM        | ×     | 18.58% ± 2.17%        | 15.70 ± 0.32       | 35.67% ± 2.67%        | 10.23 ± 0.31       |
| Rollout    | ✓     | 38.70% ± 2.09%        | 12.54 ± 0.31       | 52.93% ± 3.08%        | 10.45 ± 0.33       |
| Rollout    | ×     | 35.55% ± 1.15%        | 13.94 ± 0.46       | 54.35% ± 4.32%        | 11.13 ± 0.23       |
| RELAY (sg) | ✓     | 58.42% ± 0.11%        | 7.62 ± 0.05        | 70.75% ± 0.92%        | 6.13 ± 0.09        |
| RELAY (sg) | ×     | 59.45% ± 3.06%        | 7.59 ± 0.11        | 70.65% ± 3.18%        | 6.01 ± 0.03        |
| RELAY      | ✓     | <b>62.67%</b> ± 2.40% | <b>7.43</b> ± 0.18 | <b>76.42%</b> ± 2.47% | <b>5.86</b> ± 0.08 |
| RELAY      | ×     | <b>62.07%</b> ± 0.70% | <b>7.20</b> ± 0.18 | <b>73.10%</b> ± 2.64% | <b>5.80</b> ± 0.08 |

Table 1: Sudoku exact match and mean NFE at  $\tau = 0.15$ . *Unfiltered test* reports performance on puzzles iterated from the test split in dataset order; *deduction-only* restricts to puzzles whose solver trace requires Advanced/Master heuristics (no recursive backtracking). Accuracies are % exact match, with sample s.d. across 3 training seeds. See Appendix C.3 for the evaluation window and filter implementation.

**Base model.** As our base model, we choose Fast-dLLM v2 (1.5B parameters) [Wu et al., 2025b], a state-of-the-art DLM adapted from Qwen2.5 [Qwen et al., 2025] by finetuning on the LLaMA-Nemotron dataset [Bercovich et al., 2025].

**Training.** For RELAY adaptation we apply supervised fine-tuning to all parameters for 200 optimizer steps at effective batch size 32 on a 60,000-example mixture of filtered OpenCodeInstruct and OpenMathInstruct-2 examples with a 40/60 code/math proportion (dataset and hardware details in Appendix C.3.5). To make Algorithm 1 compatible with state-of-the-art DLMs that combine block-autoregressive decoding with KV caching, we make two careful adaptations to the on-policy rollout. First, we run the  $K=2$  relay rollout *only inside the active block* of Fast-dLLM v2’s BD3-LM-style doubled (block-causal  $\oplus$  block-bidirectional) attention [Arriola et al., 2024, Wu et al., 2025b], leaving previously decoded blocks frozen so their inter-block KV cache is reused unchanged across both passes. Second, within the active block we update the relay state  $h$  *only at positions that are still masked*: clean (already-committed) sub-block tokens contribute attention but their relay entries are not overwritten, which keeps within-block sub-block KV cache entries valid as the block fills in.

| Method                     | HumanEval    |              |             | MBPP         |              |             |
|----------------------------|--------------|--------------|-------------|--------------|--------------|-------------|
|                            | Base↑        | Plus↑        | NFE↓        | Base↑        | Plus↑        | NFE↓        |
| <b>Fast-dLLM-v2 (1.5B)</b> | 38.4%        | 35.4%        | 178.1       | 46.8%        | 39.7%        | 133.0       |
| Vanilla SFT                | 38.4%        | 34.1%        | 130.7       | 43.9%        | 38.1%        | 84.8        |
| RELAY (sg)                 | 38.4%        | 35.4%        | 104.4       | 43.1%        | 39.2%        | 80.1        |
| RELAY                      | <b>42.1%</b> | <b>37.2%</b> | <b>88.3</b> | <b>46.6%</b> | <b>41.5%</b> | <b>78.8</b> |

Table 2: Pretrained adaptation on Fast-dLLM-v2 (1.5B), evaluated at threshold 0.85. Average NFE is computed as the mean per-example count of active denoising forward calls during batched sample generation, excluding prompt prefill and final cache-update next-token forwards. Bold values are selected among adapted rows only, excluding the off-the-shelf baseline.

**Evaluation.** Inference follows Fast-dLLM v2’s confidence-based parallel decoding [Wu et al., 2025b]: within each block, the backbone applies the token-shift head so masked positions are read from the preceding token’s logit row, samples are drawn with top- $p$  filtering ( $p=0.95$ , temperature 0), and a position unmask when the probability of its sampled token exceeds a confidence threshold  $\tau$ , while the argmax masked position in each active sub-block is always unmasked so every forward makes progress. We use block length 32, sub-block length 8, and  $\tau=0.85$  for all HumanEval/MBPP numbers below (including NFE in Table 2). The *Plus* columns report HumanEval+ and MBPP+ from EvalPlus [Liu et al., 2023]—expanded unit-test suites released with the EvalPlus framework<sup>5</sup>—in the same *Base/Plus* layout used for code results in Wu et al. [2025b].

As in Sudoku, RELAY pushes the accuracy-NFE frontier here: it attains the best raw NFE among adapted methods on both HumanEval and MBPP, while also reaching the best accuracies. Notably,

<sup>5</sup><https://github.com/evalplus/evalplus>

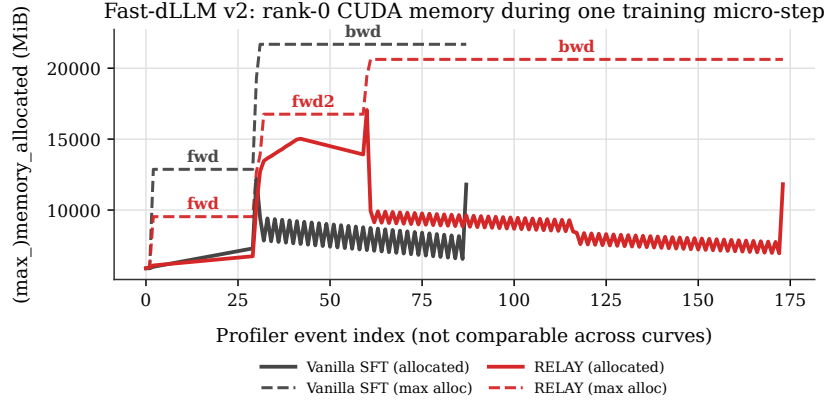


Figure 3: GPU memory during one training micro-step of Fast-dLLM v2 on an A100 80GB. Solid lines show the live GPU memory at every decoder-layer forward/backward hook. Dashed lines show the running maximum of live memory within the same micro-step (high-water mark). Phase labels (fwd, fwd2, bwd) mark each phase’s plateau. RELAY carries higher live memory through fwd2, but its peak ( $\approx 20.1$  GiB) lands within  $\approx 1$  GiB of vanilla SFT’s ( $\approx 21.2$  GiB); see main text and Appendix C.4 for the mechanism.

234 **on HumanEval, RELAY even surpasses the vanilla SFT accuracy at 32% less NFEs** (88.3 vs.  
 235 130.7), demonstrating that the RELAY improves both accuracy and the number of denoising steps  
 236 required to reach it.

#### 237 4.2.1 Training memory overhead

238 A natural concern is that BPTT through  $K=2$  forward passes inflates training memory. Figure 3  
 239 profiles one micro-step on an A100 80GB. Each regime is shown with two curves: the solid trace  
 240 samples live GPU memory at every transformer-layer hook, and the dashed trace is its running  
 241 maximum, a high-water mark whose final value is the peak the run actually demanded. The dashed  
 242 trace sits above the solid trace whenever a transient allocation falls between hooks. The largest  
 243 such transient—and the binding peak of the whole micro-step in both regimes—is the cross-entropy  
 244 backward through the vocabulary-projection head (lm\_head), which materializes a  $B \times T \times V$  fp32  
 245 grad-of-logits buffer at the start of bwd.

246 RELAY’s second forward raises the live trace by  $\approx 5$  GiB through fwd2: the saved activations of  
 247 forward 1 and the relay state  $h$  coexist with forward 2 to route credit through both passes (Algorithm 1).  
 248 Most of that elevation is autograd intermediates rather than saved-for-backward state, and PyTorch  
 249 releases it in a single step before the lm\_head spike fires—live drops by  $\approx 7$  GiB for RELAY versus  
 250  $\approx 2.7$  GiB for vanilla, leaving the two regimes within  $\approx 0.5$  GiB of each other just before the spike.  
 251 Adding the spike yields nearly identical peaks, 20.1 GiB for RELAY versus 21.2 GiB for vanilla SFT,  
 252 with the residual gap on the order of allocator-timing noise. BPTT through  $K=2$  therefore does not  
 253 double peak memory in this setup (gradient checkpointing, ZeRO-3, non-fused CE), and we expect  
 254 the same whenever the lm\_head backward dominates the activation footprint. Per-phase numbers  
 255 and the profiling protocol are deferred to Appendix C.4.

## 256 5 Related Work

257 **Masked diffusion language models.** Discrete diffusion models [Austin et al., 2021] apply the  
 258 iterative denoising principles of continuous diffusion [Ho et al., 2020, Song et al., 2021] to categorical  
 259 sequences. Masked diffusion, or absorbing-state diffusion, has since become the primary framework  
 260 for language modeling within this space. While early works like MDLM [Sahoo et al., 2024] and  
 261 GenDDM [Shi et al., 2024] introduced simplified training objectives to reach competitive perplexities,  
 262 more recent models such as LLaDA [Nie et al., 2025], Dream [Ye et al., 2025], and Fast-dLLM [Wu  
 263 et al., 2025a,b] have successfully scaled the paradigm to larger models sizes.

264 **Cross-step bottlenecks in denoising.** The same discrete interface that makes masked diffusion  
 265 attractive also limits what can be communicated between denoising steps: rich internal representations



are collapsed into sampled tokens before the next step begins. Hayakawa et al. [2025] analyzes a related limitation of factored reverse transitions, showing that they may require many steps to approximate the true joint distribution and proposing mixture-model distillation as a remedy. Jo et al. [2026] frame the collapse itself as a “sampling wall” and introduce *loopholing*, a continuous pathway that carries hidden state across steps. While loopholing provides a structural mechanism for state persistence, it does not explicitly optimize the carried information for future predictions. Our paradigm extends this concept by training the relay channel end-to-end via BPTT.

**Progressive unmasking and train-test alignment.** A complementary line of work aligns the training distribution with progressive inference to fix the mismatch caused by standard uniform mask sampling. PUMA [Kim et al., 2026] addresses this using a streaming buffer of partially denoised examples, while PAPL [Bezemek et al., 2026] incorporates planner dynamics. These methods improve the sequence of discrete states the model trains on; our work complements this by exploring what continuous information can be carried alongside those states.

**Latent state, memory, and self-conditioning.** Several recent approaches augment discrete generation with continuous variables or a persistent state. LADD [Shariatian et al., 2026] jointly diffuses tokens and auxiliary latents through an ELBO; CADD [Zheng et al., 2025] pairs each position with a continuous diffusion process; VADD [Xie et al., 2025] trains a VAE atop discrete diffusion. Related ideas also appear outside diffusion: Latent Thought Models [Kong et al., 2025] introduce thought vectors for autoregressive models, and He et al. [2026] study “latent tokens” whose distributions are predicted but not decoded. MetaState [Xia et al., 2026] adds a fixed-size working memory ( $\sim 70\text{M}$  params) to frozen dLLMs and trains it over multi-step denoising rollouts. DRAKES [Wang et al., 2025b] instead backpropagates through discrete diffusion trajectories with a Gumbel-Softmax relaxation [Jang et al., 2017]. These works show several ways of giving a generator more than hard tokens to work with, either through auxiliary latents or differentiable trajectory objectives.

**Prediction-space feedback.** Self-conditioning [Chen et al., 2023] and Soft-Masked Diffusion [Hersche et al., 2025] provide a lighter-weight form of cross-step feedback by feeding previous output distributions or top- $k$  predictions back into the model. These methods reuse predictions from earlier steps, offering another route for soft information to persist across the discrete sampling interface.

## 6 Conclusion

We introduced RELAY to address the hard reset problem in MDMs by passing a continuous, differentiable latent state across inference steps. By training this relay channel via truncated BPTT, we demonstrated that discrete diffusion models can explicitly optimize intermediate representations for future unmasking decisions, advancing the performance-latency Pareto frontier.

**Limitations.** RELAY introduces specific computational trade-offs. First, the relay mechanism incurs a per-step computational overhead associated with reading and writing the continuous state  $h$ . However, because the model requires fewer overall forward passes to reach the same accuracy, this can still yield a net improvement in end-to-end wall-clock time. Second, performing two-step BPTT during training can double the activation memory requirements compared to a standard single pass. However, as we show in our memory analysis, the combination of sharding and gradient checkpointing can effectively get rid of this memory overhead.

## References

- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 17981–17993, 2021.
- Andrew Campbell, Joe Benton, Valentin De Bortoli, Tom Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://openreview.net/forum?id=DmT862YAieY>.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models, 2024. URL <http://arxiv.org/abs/2406.07524>.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data, 2024. URL <http://arxiv.org/abs/2406.04329>.
- Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. Can looped transformers learn to implement multi-step gradient descent for in-context learning? In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 15130–15152. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/gatmiry24b.html>.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *The Twelfth International Conference on Learning Representations*, 2024.
- Marianne Arriola, Subham Sekhar Sahoo, Yair Schiff, Alexander Rush, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models, 2024. URL <http://arxiv.org/abs/2503.09573>.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025a.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm, 2025b. URL <https://arxiv.org/abs/2509.26328>.
- Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=CTC7CmirNr>.
- Shen Nie, Fengqi Zhu, Chao You, Xiaojie Zhang, Jingyang Ou, and Jun Zhu. LLaDA: Large language diffusion with autoregressive initialization, 2025. URL <http://arxiv.org/abs/2502.09992>.
- Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated Sampling from Masked Diffusion Models via Entropy Bounded Unmasking, May 2025. URL <http://arxiv.org/abs/2505.24857>.
- Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham M. Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=DjJmre5IkP>.

354 Dhruvesh Patel, Tahira Naseem, Gaurav Pandey, Md Arafat Sultan, Andrew McCallum, and  
355 Ramón Fernandez Astudillo. Improved sampling from masked diffusion models with position con-  
356 trastive guidance. In *NeurIPS 2025 Workshop on Structured Probabilistic Inference & Generative*  
357 *Modeling*, 2025. URL <https://openreview.net/forum?id=eOWmOrWbtc>.

358 Gary McGuire, Bastian Tugemann, and Gilles Civario. There is no 16-clue sudoku: Solving the  
359 sudoku minimum number of clues problem via hitting set enumeration. *Experimental Mathematics*,  
360 23(2):190–217, 2014.

361 Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and  
362 Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025a.

363 Jaeyeon Kim, Mingyu Jo, and Seyoung Yun. Stop training for the worst: Progressive unmasking  
364 accelerates masked diffusion training, 2026. URL <http://arxiv.org/abs/2602.10314>.

365 Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan  
366 Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang,  
367 Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin  
368 Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi  
369 Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan,  
370 Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL  
371 <https://arxiv.org/abs/2412.15115>.

372 Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil,  
373 Zach Moshe, Tomer Ronen, Najeeb Nabwani, et al. Llama-nemotron: Efficient reasoning models.  
374 *arXiv preprint arXiv:2505.00949*, 2025.

375 Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by  
376 ChatGPT really correct? rigorous evaluation of large language models for code generation. In  
377 *Advances in Neural Information Processing Systems*, 2023. URL [https://openreview.net/](https://openreview.net/forum?id=1qvx610Cu7)  
378 [forum?id=1qvx610Cu7](https://openreview.net/forum?id=1qvx610Cu7).

379 Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL  
380 <http://arxiv.org/abs/2006.11239>.

381 Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben  
382 Poole. Score-based generative modeling through stochastic differential equations, 2021. URL  
383 <http://arxiv.org/abs/2011.13456>.

384 Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng  
385 Kong. Dream 7b: Diffusion large language models, 2025. URL [https://arxiv.org/abs/](https://arxiv.org/abs/2508.15487)  
386 [2508.15487](https://arxiv.org/abs/2508.15487).

387 Satoshi Hayakawa, Yuhta Takida, Masaaki Imaizumi, Hiromi Wakaki, and Yuki Mitsufuji. Distillation  
388 of discrete diffusion through dimensional correlations, 2025. URL [http://arxiv.org/abs/](http://arxiv.org/abs/2410.08709)  
389 [2410.08709](http://arxiv.org/abs/2410.08709).

390 Mingyu Jo, Jaeyeon Kim, and Seyoung Yun. Loopholing discrete diffusion: Deterministic bypass of  
391 the sampling wall, 2026. URL <http://arxiv.org/abs/2510.19304>.

392 Zachary Bezemek, Pengzhang Zhang, Joey Bose, et al. Planner aware path learning in diffusion  
393 language models training, 2026. URL <http://arxiv.org/abs/2509.23405>.

394 Dario Shariatian, Alain Durmus, Umut Simsekli, and Stefano Peluchetti. Latent-augmented discrete  
395 diffusion models, 2026. URL <http://arxiv.org/abs/2510.18114>.

396 Huangjie Zheng, Shansan Gong, Ruixiang Zhang, Tianrong Chen, Jiatao Gu, Mingyuan Zhou,  
397 Navdeep Jaitly, and Yizhe Zhang. Continuously augmented discrete diffusion model for categorical  
398 generative modeling, 2025. URL <http://arxiv.org/abs/2510.01329>.

399 Tianyu Xie, Shuchen Xue, Zijin Feng, Tianyang Hu, Jiacheng Sun, Zhenguo Li, and Cheng Zhang.  
400 Variational autoencoding discrete diffusion with enhanced dimensional correlations modeling,  
401 2025. URL <http://arxiv.org/abs/2505.17384>.

402 Deqian Kong, Minglu Zhao, Dehong Xu, Bo Pang, Shu Wang, Edouardo Honig, Zhangzhang Si,  
403 Chuan Li, Jianwen Xie, Sirui Xie, and Ying Nian Wu. Latent thought models with variational  
404 bayes inference-time computation, 2025. URL <http://arxiv.org/abs/2502.01567>.

405 Andre He, Sean Welleck, and Daniel Fried. Reasoning with latent tokens in diffusion language  
406 models, 2026. URL <http://arxiv.org/abs/2602.03769>.

407 Kejing Xia et al. MetaState: Persistent working memory for discrete diffusion language models,  
408 2026. URL <http://arxiv.org/abs/2603.01331>.

409 Chenyu Wang, Masatoshi Uehara, Yichun He, Amy Wang, Tommaso Biancalani, Avantika Lal,  
410 Tommi Jaakkola, Sergey Levine, Hanchen Wang, and Aviv Regev. Fine-tuning discrete diffusion  
411 models via reward optimization with applications to DNA and protein design, 2025b. URL  
412 <http://arxiv.org/abs/2410.13643>.

413 Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-softmax. In  
414 *International Conference on Learning Representations (ICLR)*, 2017.

415 Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog bits: Generating discrete data using  
416 diffusion models with self-conditioning, 2023. URL <http://arxiv.org/abs/2208.04202>.

417 Michael Hersche, Aditya Ganeshan, Robin Bartoszcze, Robert Gerstenberger, Luca Benini, and  
418 Abbas Raghunathan. Soft-masked diffusion language models, 2025. URL <http://arxiv.org/abs/2510.17206>.

420 Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and LingPeng Kong. Generative recursive  
421 reasoning models. In *International Conference on Learning Representations (ICLR)*, 2025. URL  
422 <https://openreview.net/pdf?id=Vxu6kcIjwV>.

423 Tim Vink. sudoku-solver: a python Sudoku solver that traces the human-style strategies it uses.  
424 <https://github.com/timvink/sudoku-solver>, 2024.

|     |                                                                            |           |
|-----|----------------------------------------------------------------------------|-----------|
| 425 | <b>Appendix</b>                                                            |           |
| 426 | <b>A Summary of Notation</b>                                               | <b>14</b> |
| 427 | <b>B Proofs</b>                                                            | <b>15</b> |
| 428 | <b>C Experimental Details</b>                                              | <b>16</b> |
| 429 | C.1 Model Architectures . . . . .                                          | 16        |
| 430 | C.1.1 Sudoku model . . . . .                                               | 16        |
| 431 | C.1.2 Fast-dLLM v2 . . . . .                                               | 17        |
| 432 | C.2 Training Hyperparameters . . . . .                                     | 17        |
| 433 | C.2.1 Sudoku experiments (300k steps) . . . . .                            | 17        |
| 434 | C.3 Dataset Details . . . . .                                              | 17        |
| 435 | C.3.1 Sudoku Extreme . . . . .                                             | 17        |
| 436 | C.3.2 Deduction-only cohort . . . . .                                      | 18        |
| 437 | C.3.3 Evaluation protocol for Table 1 . . . . .                            | 18        |
| 438 | C.3.4 OpenCode/OpenMath c40m60 mixture for Fast-dLLM v2 adaption . . . . . | 18        |
| 439 | C.3.5 Fast-dLLM v2 training hardware and parallelism . . . . .             | 18        |
| 440 | C.4 Fast-dLLM v2 memory profiling . . . . .                                | 18        |

## 441 A Summary of Notation

442 Table 3 summarizes the notation used throughout the paper.

| Notation                                 | Meaning                                                                                                                        |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| $L$                                      | Sequence length.                                                                                                               |
| $d$                                      | Model dimension.                                                                                                               |
| $[M]$                                    | Mask token.                                                                                                                    |
| $\mathcal{V}$                            | Vocabulary over tokens, which includes $[M]$ .                                                                                 |
| $\mathcal{X} := \mathcal{V}^n$           | Space of sequences over vocabulary.                                                                                            |
| $x_0 \in \mathcal{X}$                    | Clean target sequence.                                                                                                         |
| $z$                                      | Partially masked sequence.                                                                                                     |
| $\mathbf{h} \in \mathbb{R}^{n \times d}$ | Hidden state which serves as (optionally pre-transform) relay representation.                                                  |
| $\tilde{\mathbf{h}}$                     | Relay input at step $t$ (defaults to $\mathbf{h}_{\text{out}}$ ; no-BPTT ablation uses $\text{sg}[\mathbf{h}_{\text{out}}]$ ). |
| $\mathcal{B}$                            | Rollout buffer storing each example’s current token state, clean target, and latent state across optimizer steps.              |
| $\text{EMB}_{\theta}$                    | Token embedding map.                                                                                                           |
| $\text{UNEMB}_{\theta}$                  | Learned readout/unembedding head mapping hidden states to token logits.                                                        |
| $R_{\theta}(\mathbf{h}, z)$              | Relay from hidden state and discrete state to the next latent state.                                                           |
| $\tau$                                   | Confidence threshold for choosing which masked positions update.                                                               |
| $\mathcal{U}_t$                          | Positions selected for update at step $t$ .                                                                                    |
| CE                                       | Cross-entropy loss.                                                                                                            |
| $\text{sg}[\cdot]$                       | Stop gradient.                                                                                                                 |
| $\mathcal{M}_{full}$                     | Full $2L \times 2L$ attention mask for concatenated noised and clean sequences.                                                |
| $\mathcal{M}_{BD}$                       | Mask component for intra-block bidirectional refinement.                                                                       |
| $\mathcal{M}_{OBC}$                      | Mask component for attending to the clean prefix.                                                                              |
| $\mathcal{M}_{BC}$                       | Mask component enforcing standard left-to-right causality.                                                                     |
| $x_0^{<b}$                               | Clean, denoised prefix blocks saved as read-only context.                                                                      |
| $x_t^b$                                  | Active, partially noised block.                                                                                                |
| $m, \dot{m}$                             | Complementary masks ( $1 - m$ ) to train both views in a single pass.                                                          |

Table 3: Notation



## B Proofs

The relay state makes the rollout generally non-Markovian in the masked sequence  $\mathbf{x}_{t_k}$  alone. However, it is Markovian in the augmented state  $\mathbf{s}_k = (\mathbf{x}_{t_k}, \mathbf{h}_{t_k})$ . We therefore state the rollout guarantees on the augmented state space.

Fix a time grid  $t_0 > \dots > t_K$  and write the augmented state as  $\mathbf{s}_k = (\mathbf{x}_{t_k}, \mathbf{h}_{t_k})$ . The initial state  $\mathbf{s}_0 = (\{\text{[M]}\}^L, \mathbf{h}_{t_0})$  is independent of  $\mathbf{x}_0$ . At step  $k$ , the model computes logits and the next relay state from the current augmented state,

$$(\ell_k, \bar{\mathbf{h}}_{k+1}) = H_\theta(\mathbf{s}_k), \quad (9)$$

and the unmasking policy selects a single masked position using only the current logits, partially masked sequence, and fresh policy randomness,

$$I_k \sim u(\cdot \mid \ell_k, \mathbf{x}_{t_k}). \quad (10)$$

The policy randomness is independent of  $\mathbf{x}_0$  conditional on  $\mathbf{s}_k$ . After selecting  $I_k$ , the next masked sequence is obtained by teacher forcing,

$$x_{t_{k+1}}^i = \begin{cases} x_0^i, & i = I_k, \\ x_{t_k}^i, & i \neq I_k, \end{cases} \quad \mathbf{s}_{k+1} = (\mathbf{x}_{t_{k+1}}, \bar{\mathbf{h}}_{k+1}). \quad (11)$$

Thus hidden entries of  $\mathbf{x}_0$  influence future computation only after they have been revealed in  $\mathbf{x}_{t_k}$ .

**Proposition 1** (Augmented teacher-forcing agreement). *Suppose the adapted rollout condition holds, and fix the rollout mechanism  $H_\theta$  and unmasking policy  $u$ . Consider two augmented chains initialized from the same distribution. In the teacher-forced chain, sample  $\mathbf{x}_0 \sim p_{\text{data}}$  once, select  $I_k$  according to  $u(\cdot \mid \ell_k, \mathbf{x}_{t_k})$ , and reveal the corresponding ground-truth token  $X_0^{I_k}$ . In the idealized augmented inference chain, at the same augmented state  $\mathbf{s}_k$ , select  $I_k$  according to the same policy, and sample the revealed token from the per-position augmented posterior*

$$V_k \sim \mathbb{P}(X_0^{I_k} = \cdot \mid \mathbf{s}_k). \quad (12)$$

The relay update is the same function of the current augmented state in both chains. Then, for every  $k$ , the marginal law of the augmented state  $\mathbf{s}_k$  under the teacher-forced chain equals the marginal law of  $\mathbf{s}_k$  under the idealized augmented inference chain.

*Proof.* We show that the two augmented chains have the same one-step transition kernel. The initial augmented state has the same distribution in both chains by assumption. Fix a step  $k$  and condition on the current augmented state  $\mathbf{s}_k = \mathbf{s} = (\mathbf{z}, \mathbf{h})$ . The logits and relay update are then fixed as

$$(\ell, \bar{\mathbf{h}}) = H_\theta(\mathbf{s}). \quad (13)$$

Both chains select the revealed position from the same conditional distribution

$$I \sim u(\cdot \mid \ell, \mathbf{z}). \quad (14)$$

Moreover, by the adapted rollout condition, the policy randomness is independent of  $\mathbf{x}_0$  conditional on  $\mathbf{s}$ . Therefore, after additionally conditioning on the realized revealed position  $I = i$ , the teacher-forced token has conditional law

$$\mathbb{P}(X_0^i = \cdot \mid \mathbf{s}_k = \mathbf{s}, I = i) = \mathbb{P}(X_0^i = \cdot \mid \mathbf{s}_k = \mathbf{s}). \quad (15)$$

This is exactly the token law used by the idealized augmented inference chain. Thus, for any token value  $a \in \mathcal{V}$ , both chains assign the same conditional probability to the next augmented state

$$\mathbf{s}_{k+1} = (\mathbf{z}^{(i \leftarrow a)}, \bar{\mathbf{h}}), \quad (16)$$

namely

$$u(i \mid \ell, \mathbf{z}) \mathbb{P}(X_0^i = a \mid \mathbf{s}_k = \mathbf{s}). \quad (17)$$

Summing over all positions and token values that produce a given next augmented state, the two transition kernels coincide. Since the initial distributions also coincide, induction on  $k$  gives equality of the marginal law of  $\mathbf{s}_k$  for every  $k$ .  $\square$

477 **Proposition 2** (Augmented posterior minimizer). *Fix an augmented rollout distribution satisfying*  
 478 *the following non-leakage factorization: for every augmented state  $\mathbf{s} = (\mathbf{z}, \mathbf{h})$  reachable at time  $t_k$ ,*

$$\mathbb{P}(\mathbf{s}_k = \mathbf{s} \mid \mathbf{x}_0) = \alpha_k(\mathbf{s}) \mathbf{1}\{\mathbf{x}_0^{[L] \setminus \mathcal{M}(\mathbf{z})} = \mathbf{z}^{[L] \setminus \mathcal{M}(\mathbf{z})}\}, \quad (18)$$

479 *for some nonnegative function  $\alpha_k$  that does not depend on the unrevealed entries of  $\mathbf{x}_0$ . Consider the*  
 480 *augmented cross-entropy risk*

$$\mathcal{L}(q) = \mathbb{E} \left[ \sum_{k=0}^{K-1} \sum_{i: \mathbf{x}_{t_k}^i = [M]} -\log q^i(X_0^i \mid \mathbf{s}_k) \right], \quad (19)$$

481 *where  $q^i(\cdot \mid \mathbf{s})$  ranges over conditional distributions on the vocabulary. Then any pointwise minimizer*  
 482 *satisfies*

$$q^{i,*}(\cdot \mid \mathbf{s}) = \mathbb{P}(X_0^i = \cdot \mid \mathbf{s}) \quad (20)$$

483 *for every masked coordinate  $i$  and every augmented state  $\mathbf{s}$  with positive probability. Moreover,*  
 484 *Bayes' rule gives*

$$\mathbb{P}(\mathbf{x}_0 \mid \mathbf{s} = (\mathbf{z}, \mathbf{h})) \propto \alpha_k(\mathbf{z}, \mathbf{h}) \mathbf{1}\{\mathbf{x}_0^{[L] \setminus \mathcal{M}(\mathbf{z})} = \mathbf{z}^{[L] \setminus \mathcal{M}(\mathbf{z})}\} p_{\text{data}}(\mathbf{x}_0). \quad (21)$$

485 *Thus the target is the Bayes-optimal posterior given the information available to the augmented*  
 486 *inference process.*

487 *Proof.* We show that the two augmented chains have the same one-step transition kernel. The initial  
 488 augmented state has the same distribution in both chains by assumption. Fix a step  $k$  and condition  
 489 on the current augmented state  $\mathbf{s}_k = \mathbf{s} = (\mathbf{z}, \mathbf{h})$ . The logits and relay update are then fixed as

$$(\boldsymbol{\ell}, \bar{\mathbf{h}}) = H_{\theta}(\mathbf{s}). \quad (22)$$

490 Both chains select the revealed position from the same conditional distribution

$$I \sim u(\cdot \mid \boldsymbol{\ell}, \mathbf{z}). \quad (23)$$

491 Moreover, by the adapted rollout condition, the policy randomness is independent of  $\mathbf{x}_0$  conditional  
 492 on  $\mathbf{s}$ . Therefore, after additionally conditioning on the realized revealed position  $I = i$ , the teacher-  
 493 forced token has conditional law

$$\mathbb{P}(X_0^i = \cdot \mid \mathbf{s}_k = \mathbf{s}, I = i) = \mathbb{P}(X_0^i = \cdot \mid \mathbf{s}_k = \mathbf{s}). \quad (24)$$

494 This is exactly the token law used by the idealized augmented inference chain. Thus, for any token  
 495 value  $a \in \mathcal{V}$ , both chains assign the same conditional probability to the next augmented state

$$\mathbf{s}_{k+1} = \left( \mathbf{z}^{(i \leftarrow a)}, \bar{\mathbf{h}} \right), \quad (25)$$

496 namely

$$u(i \mid \boldsymbol{\ell}, \mathbf{z}) \mathbb{P}(X_0^i = a \mid \mathbf{s}_k = \mathbf{s}). \quad (26)$$

497 Summing over all positions and token values that produce a given next augmented state, the two  
 498 transition kernels coincide. Since the initial distributions also coincide, induction on  $k$  gives equality  
 499 of the marginal law of  $\mathbf{s}_k$  for every  $k$ .  $\square$

## 500 C Experimental Details

### 501 C.1 Model Architectures

#### 502 C.1.1 Sudoku model

503 We use a small rotary Transformer with 4 layers, hidden dimension  $d=128$ , 4 attention heads,  
 504 feedforward dimension  $4d=512$ , dropout 0.1, ReLU activation, and rotary embedding dimension 64.  
 505 The loop-holing variant adds a LayerNorm for hidden state injection and reads the carry from the final  
 506 encoder layer (layer index  $-1$ ). Total parameters: approximately 1.5M.

## 507 C.1.2 Fast-dLLM v2

508 Unlike standard MDMs, which denoise the entire token sequence globally, Fast-dLLM v2 models  
 509 [Wu et al., 2025b] a block-wise Markov process. By partitioning the sequence into blocks of size  
 510  $D$ , it targets the local conditional distribution  $p_\theta(x^b|x_t^b, x_0^{<b})$ . This localizes the diffusion process  
 511 while anchoring it to an autoregressive prefix, successfully bypassing the immense pretraining costs  
 512 associated with full-attention MDMs.

513 The core architectural shift lies in its attention topology. Fast-dLLM v2 concatenates the noised  $x_t$  and  
 514 clean  $x_0$  sequences into a  $2L$ -length tensor, governed by a full attention mask  $\mathcal{M}_{full} \in \{0, 1\}^{2L \times 2L}$   
 515 [Arriola et al., 2024]. This mask explicitly splits into three distinct functional roles:

- 516 •  $\mathcal{M}_{BD}$ : Enables intra-block bidirectional refinement for the currently active block.
- 517 •  $\mathcal{M}_{OBC}$ : Allows the active block to attend to the completely denoised, clean prefix  $x_0^{<b}$ .
- 518 •  $\mathcal{M}_{BC}$ : Enforces standard left-to-right causality across blocks.

519 A complementary masking strategy ( $m$  and  $\bar{m} = 1 - m$ ) effectively trains both the noised and clean  
 520 views simultaneously in a single forward pass.

521 At inference, this topology enables hierarchical Key-Value caching—a major advantage over standard  
 522 MDMs, which typically require full-sequence recomputation at every denoising step. Completely  
 523 denoised blocks  $x_0^{<b}$  are saved as read-only context, while a DualCache handles prefix and suffix  
 524 activations within the active, partially noised block  $x_t^b$ .

## 525 C.2 Training Hyperparameters

### 526 C.2.1 Sudoku experiments (300k steps)

527 We utilize the following hyperparameters:

- 528 • Batch size: 512
- 529 • Optimizer: AdamW,  $\text{lr} = 5 \times 10^{-4}$
- 530 • LR schedule: constant with 2k warmup steps
- 531 • Gradient clipping: 0.5
- 532 • BPTT unroll depth:  $T = 2$
- 533 • Confidence threshold:  $\tau = 0.15$  (with  $\sigma = 0.1$  perturbation during training)
- 534 • Confidence metric: `top_prob` (maximum softmax probability)
- 535 • Validation: every 5k steps, 100 batches, threshold sweep over  $\{0.05, 0.10, 0.15, 0.20, 0.25\}$
- 536 • Hardware:  $1 \times \text{GPU (A100 40GB or A6000)}$

## 537 C.3 Dataset Details

### 538 C.3.1 Sudoku Extreme

539 The Sudoku-Extreme dataset [Gong et al., 2025] consists of  $9 \times 9$  Sudoku puzzles with 17 given clues,  
 540 the minimum number compatible with a uniquely solvable Sudoku puzzle [McGuire et al., 2014].  
 541 Each puzzle is represented as a flat sequence of length  $L=81$  (one token per cell, row-major order)  
 542 over a vocabulary of  $|\mathcal{V}|=11$  task tokens: digits  $\{1, \dots, 9\}$ , a blank/zero token for unfilled cells,  
 543 and a mask token. The 17 clue positions are treated as fixed and are not modified during inference;  
 544 the remaining 64 positions are mutable. Standard Sudoku constraints apply: each digit 1–9 must  
 545 appear exactly once in each row, column, and  $3 \times 3$  box. We use the original training split, which  
 546 contains 3,831,994 puzzles, as a 32-shard iterable dataset with no additional shuffle buffer, making  
 547 the training order deterministic for a fixed cache and distributed layout. For validation, we use the  
 548 original test split, which contains 422,786 puzzles, but evaluate only the first 100 batches at batch  
 549 size 512 (51,200 puzzles) at each checkpoint. The validation dataloader is not shuffled, and this fixed  
 550 prefix is comparable in difficulty to the rest of the test split under the dataset’s provided rating field  
 551 (mean/median rating 21.7/16 for the prefix versus 22.2/17 for the remaining examples). This task is  
 552 a strong test for cross-step coordination because satisfying all constraints simultaneously requires  
 553 global reasoning—placing a digit in one cell constrains all cells in its row, column, and box.

### 554 C.3.2 Deduction-only cohort

555 For the qualitative legality analysis of Section 4.1 (where RELAY differs from RELAY (sg)) we  
556 annotate each example of Sudoku-Extreme using Vink [2024]’s Sudoku solver<sup>6</sup>, which records the  
557 human-like deduction strategies used to solve each puzzle and falls back on recursive backtracking  
558 when those fail. Since the studied architectures cannot perform recursive search, we keep only  
559 puzzles whose solver trace contains *Advanced* (Naked Pair, Hidden Pair, Naked Triple, Hidden Triple,  
560 Naked Quad, Hidden Quad) or *Master* (X-Wing, Swordfish, Jellyfish, Forcing Chain) strategies and  
561 never falls back on recursive backtracking. The resulting cohort contains 2,000 test puzzles (1,933  
562 Advanced + 67 Master).

### 563 C.3.3 Evaluation protocol for Table 1

564 Each cell of Table 1 aggregates the first  $N=2000$  puzzles yielded under our offline trajectory-  
565 dump script’s caps (MAX\_EXAMPLES/LIMIT\_VAL\_BATCHES); this is a strict prefix of the  $\sim 422k$ -  
566 row Hugging Face test split iterated in dataset order, rather than a separately held-out val-  
567 idation split. Per-seed sample sizes match across methods, but the deduction-only filter  
568 (`sudoku_extreme_deduction_only_filter_fn`, which keeps only puzzles whose solver trace  
569 requires Advanced or Master heuristics and never falls back on recursive backtracking) can change  
570 which puzzles appear within that 2000-puzzle window.

### 571 C.3.4 OpenCode/OpenMath c40m60 mixture for Fast-dLLM v2 adaption

572 For Fast-dLLM v2 adaptation, we construct a 60k-example supervised fine-tuning mixture from  
573 `nvidia/OpenCodeInstruct`<sup>7</sup> and `nvidia/OpenMathInstruct-2`<sup>8</sup>. The code portion contains  
574 24,000 examples from the OpenCodeInstruct training split, and the math portion contains 36,000  
575 examples from the OpenMathInstruct-2 `train_1M` split, giving a 40/60 code/math balance. Both  
576 sources are loaded in streaming mode and shuffled before filtering (buffer sizes 100k and 50k,  
577 respectively; seeds 42 and 43). Code examples are kept only when they contain a nonempty  
578 prompt and answer, include a Python `def` in the answer, have unit tests, achieve average test  
579 score 1.0, and have no failing recorded test executions. Math examples are kept only when the  
580 problem and solution are nonempty and below 4,000 and 6,000 characters, respectively. For both  
581 sources, prompts are deduplicated after whitespace/case normalization, examples containing held-  
582 out evaluation markers (`humaneval`, `mbpp`, `evalplus`, and `math-500`) are removed, and the final  
583 Fast-dLLM-v2 chat-formatted sequence must fit within 2048 tokens. The accepted examples are  
584 converted to one-turn user/assistant conversations, shuffled with seed 42, and written to the local  
585 `data/opencode_openmath_60k_c40m60/train_conversation` shards used by all Fast-dLLM  
586 v2 runs. We finetune *all* Fast-dLLM v2 parameters for 200 optimizer steps on this mixture. For  
587 RELAY on-policy rollouts during training and evaluation we use decoding threshold  $\tau=0.85$ , restrict  
588 unmasking to the active diffusion block of size 32, and apply relay updates only at positions that are  
589 still masked (updates outside the active block are not defined by the backbone).

### 590 C.3.5 Fast-dLLM v2 training hardware and parallelism

591 All adaptation runs use DeepSpeed ZeRO-3 with bf16 mixed precision and gradient checkpointing on  
592 two NVIDIA A100 80GB GPUs, with per-device batch size 2 and gradient accumulation 16 (effective  
593 batch size 32). For wall-clock characterization we logged longer reference runs over the same mixture  
594 for two epochs (1,248 global steps, matched batch and parallelism), yielding a total `train_runtime`  
595 of 9,474 s for vanilla SFT versus 42,674 s for RELAY ( $\approx 4.5\times$ ); under a linear-in-steps approximation  
596 this scales to  $\approx 1.5$  ks vs.  $\approx 6.8$  ks for the 200-step budget reported in the main text.

### 597 C.4 Fast-dLLM v2 memory profiling

598 This section gives the protocol and per-phase numbers behind Figure 3.

599 **Setup.** We profile a single training micro-step of Fast-dLLM v2 on the OpenCode/OpenMath  
600 c40m60 mixture under the same hardware and parallelism as the main runs (Section C.3.5): two

<sup>6</sup><https://github.com/timvink/sudoku-solver>

<sup>7</sup><https://huggingface.co/datasets/nvidia/OpenCodeInstruct>

<sup>8</sup><https://huggingface.co/datasets/nvidia/OpenMathInstruct-2>

601 A100 80GB GPUs, DeepSpeed ZeRO-3, bf16, and gradient checkpointing, with sequence length  
 602 2048 and per-device batch size 2. Production runs use gradient accumulation 16; profiling forces  
 603 accumulation to 1 and replaces `optimizer.step` with a no-op so that the recorded peak is attributable  
 604 to a single forward/backward pair rather than to optimizer-state allocation.

605 **Instrumentation.** On every decoder-layer forward and backward hook we record  
 606 `torch.cuda.memory_allocated` (the solid traces in Figure 3) and the running  
 607 `torch.cuda.max_memory_allocated` (the dashed traces). The dashed series is a high-  
 608 water mark maintained internally by PyTorch’s caching allocator: it is updated whenever a new  
 609 allocation makes the total live tensor footprint exceed the previous all-time maximum, and never  
 610 decreases until `reset_peak_memory_stats()` is called. We record this metric in eager mode—no  
 611 `torch.compile`, FlashAttention 2 disabled—so the steps in the dashed roof correspond directly to  
 612 discrete kernel-level allocations rather than to compiler graph fusions. Phase boundaries (fwd, fwd2,  
 613 bwd) are inferred from hook ordering and labelled at each phase’s maximum plateau. Horizontal axes  
 614 are profiler event indices (88 events for vanilla SFT, 173 for RELAY) and are not directly comparable  
 615 across the two curves.

616 **Per-phase peaks.** The dashed all-time peaks settle at 20,618 MiB ( $\approx 20.1$  GiB) for RELAY versus  
 617 21,683 MiB ( $\approx 21.2$  GiB) for vanilla SFT, with corresponding peak-reserved values of 22,874 MiB  
 618 and 24,714 MiB. In both regimes the peak is reached at the very same operation: the first backward  
 619 hook fired after the forward returns, when the cross-entropy backward through `lm_head` transiently  
 620 allocates a  $B \times T \times V$  gradient-of-logits buffer that HuggingFace materializes in fp32 for numerical  
 621 stability ( $[2, 2048, 152064] \times 4 \text{ B} \approx 2.4 \text{ GiB}$ ) on top of the bf16 logits tensor it is differentiating.  
 622 RELAY’s second forward elevates the live trace by  $\approx 5$  GiB through `fwd2` (saved activations of  
 623 forward 1 plus the relay state  $h$  must coexist with forward 2 to provide credit through both passes;  
 624 Algorithm 1), reaching 17,057 MiB at `relay_fwd2_end`, but this is still below the 20,618 MiB peak  
 625 that the CE backward sets one event later, so the live plateau through `fwd2` does not become the  
 626 binding peak. The  $\approx 1$  GiB gap in RELAY’s favor between the two final peaks is an allocator-timing  
 627 effect: more autograd intermediates from forward 1 are released by the time the CE backward fires  
 628 than vanilla releases by the analogous event ( $\Delta \text{live} = -7,119$  MiB across this transition for RELAY  
 629 vs.  $-2,727$  MiB for vanilla, mirrored in the smaller reserved pool). Rank-1 traces reproduce both  
 630 peaks within  $< 0.1$  MiB, so Figure 3 shows only rank 0.

## NeurIPS Paper Checklist

### 1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: In the introduction and abstracts we accurately map the main contributes of the paper. We (i) describe the RELAY method, (ii) explain its innovation compared to baseline MDMs and related latent-thoughts recent work, and (iii) briefly summarize the main experimental settings and their results.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

### 2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper explicitly discusses limitations in Section 6, providing a dedicated "Limitations" paragraph. It (i) openly addresses the per-step computational overhead associated with reading and writing the continuous state, (ii) notes that performing a two-step BPTT during training effectively doubles the activation memory requirements, and (iii) recognizes the need for testing RELAY against larger models and more diverse datasets.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.



### 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: We include theoretical results in Appendix D. It provides formal proofs for Proposition 1 (Augmented teacher-forcing agreement) and Proposition 2 (Augmented posterior minimizer). We state the assumptions required for these proofs, such as defining the "adapted rollout condition" and the "non-leakage factorization"

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

### 4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Appendix C outlines the exact model architectures, including layer counts and attention topology. It also explicitly lists training hyperparameters (e.g., batch size 512, AdamW optimizer, learning rates) and precisely how the streaming buffer datasets were filtered and formulated.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

## 5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide the URL to the anonymized code, which includes guidelines on how to set up the environment and run all our experiments. We describe the construction of the dataset in detail in Appendix C.3; all base datasets used are publicly available.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

## 6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Appendix C thoroughly details the experimental settings. It specifies the data splits used (e.g. first 50k examples of the Sudoku-Extreme test split), and defines hyperparameters used (e.g. optimizer, confidence threshold, etc.).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

## 7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We report statistical significance for the Sudoku experiments. Figure 2 utilizes shaded ribbons to denote  $\pm 1$  sample standard deviation across three random training seeds. Table 1 also includes the sample standard deviation for accuracy and Mean NFE metrics. However, we do not report error bars for Fast-dLLM v2 due to computational constraints with this larger model.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

## 8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We detail the hardware used for the experiments, the training duration, and the DeepSpeed optimizations in Section 4.2.1 and in the caption of Figure 3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

## 9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The paper does not involve any human-subjects experimentation, handling of sensitive data, or anonymous information. We use openly available datasets, respect their licenses, and cite the appropriate authors.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

## 10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: Because of the technical nature of this work, we do not include a dedicated section discussing its broader societal impacts. Our focus is on methodological improvements over prior work on masked diffusion models, addressed primarily to the machine learning research community.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

## 11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: Because our work does not release new, high-risk pre-trained models or scraped datasets, we do not implement specific safeguards. Our experimental focus is entirely on methodological improvements tested on harmless, pre-existing open-source data.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

## 12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Throughout the paper, we cite the creators of all datasets, models, and algorithms that we use or take as inspiration and points of comparison. All datasets and models are openly available under either the MIT or Apache 2.0 licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, [paperswithcode.com/datasets](https://paperswithcode.com/datasets) has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

## 13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce or release any new datasets or standalone models; it relies entirely on pre-existing open-source datasets to benchmark the proposed training method.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

## 14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The research exclusively utilizes automated computational benchmarks and contains no crowdsourcing or human subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

#### 15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The study does not involve human subjects, making IRB approval unnecessary.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

#### 16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLM-based tools were not used as a core, original, or non-standard component of our research methodology.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.